

Reconstructing LLM Outputs from SSE-Equivalent Length Traces: A Cross-Model Study of Leakage and Defenses

Paweł Augustynowicz ^{1,*} and Szymon Jędrzejczak ^{1,*}

Faculty of Cybernetics, Military University of Technology, gen. Sylwestra Kaliskiego 2, 00-908 Warsaw, Poland; pawel.augustynowicz@wat.edu.pl (P.A.); szymon.jedrzejczak@wat.edu.pl (S.J.)

* Correspondence: pawel.augustynowicz@wat.edu.pl (P.A.); szymon.jedrzejczak@wat.edu.pl (S.J.)

Abstract

Large language model (LLM) assistants commonly stream responses using Server-Sent Events (SSE). Transport encryption protects payload content, but application-event sizes can still encode token lengths. We quantify this leakage under a controlled, optimistic observation model in which an instrumented service exposes one SSE application event per generated token. These laboratory traces are SSE-equivalent in framing and granularity; they are not TLS, HTTP/2, or QUIC records recovered from a network capture. A fixed T5-based reconstructor was evaluated on 300 aligned prompts for each of seven open-weight models (2100 samples). Mean term-frequency cosine similarity varied from 0.324 for Qwen2.5-1.5B-Instruct to 0.024 for Phi-3.5-mini-instruct; 19 of 21 paired model comparisons remained significant after Holm correction. These results measure transfer of one fixed reconstructor and should not be interpreted as intrinsic model vulnerability. On 30 paired prompts for each of two victim models, five defense configurations based on bucketing, fixed-width framing, batching, and randomized padding reduced mean lexical recovery by 35.0–90.3% against the frozen, non-adaptive attacker. No defended sample exceeded the descriptive $\phi > 0.5$ threshold, although the pilot sample is too small to establish zero residual risk. The results demonstrate that token-granular application-length traces contain model-dependent lexical information and that simple framing transformations can reduce its utility. Passive extraction from encrypted transport traffic and adaptive attacks remain open evaluation requirements.

Keywords: large language models; side-channel analysis; server-sent events; traffic analysis; privacy leakage; token streaming; text reconstruction; AI assistants

1. Introduction

Interactive LLM assistants typically deliver responses incrementally, allowing users to see generated text almost immediately rather than waiting for a complete answer. A common implementation relies on Server-Sent Events (SSE), where each generated token is wrapped as an application-layer event of the form `data: <token>\n\n`. Transport encryption conceals event contents, but it does not automatically conceal event sizes. The present study does not recover those sizes from TLS, HTTP/2, or QUIC records. Instead, it records SSE-equivalent application events directly from an instrumented service, with exactly one event per generated token, in order to isolate the information carried by token-granular length sequences.

Our threat model builds on the line of research introduced by Weiss *et al.*, who demonstrated that streamed LLM responses may leak information through observable

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Electronics* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

token-length patterns [1]. Rather than attempting a full end-to-end network attack, we focus on a controlled setting that isolates the information contained in application-layer length traces. This allows us to quantify the leakage potential itself before addressing the additional challenges introduced by transport protocols, encryption mechanisms, and network infrastructure.

The study addresses three research questions:

- RQ1:** How much lexical information can a fixed Weiss-style reconstructor recover from SSE-equivalent one-token-per-event length traces across victim models?
- RQ2:** Are cross-model differences statistically distinguishable after prompt pairing and family-wise error correction?
- RQ3:** How effectively do bucketing, fixed framing, batching, and randomized padding reduce recovery, and at what byte and frame-count cost?

This work makes three principal contributions. First, it introduces a controlled and reproducible methodology for collecting token-level application traces from streaming LLM systems. Second, it provides a cross-model evaluation involving seven open-weight models under identical reconstruction conditions, highlighting substantial differences in leakage behavior. Third, it examines several practical mitigation strategies and quantifies the trade-offs between leakage reduction and operational overhead. The complete artifact package, including code, prompts, per-sample outputs, analysis scripts, and the reproducibility manifest, is publicly available at <https://github.com/jackaugustyn/LLeakM>.

2. Related Work

Side-channel attacks infer sensitive information from indirect observables such as timing and message size. Classical timing attacks showed that implementation-level leakage can defeat cryptographic protection without recovering plaintext directly [2]. Encrypted network applications likewise leak through packet sizes and timing. Keystroke inference, web-application traffic analysis, and website fingerprinting demonstrate that encryption does not hide communication shape [3–6].

Traffic-analysis defenses transform that shape through padding, morphing, batching, or cover traffic. Traffic morphing attempts to make one traffic class resemble another, while subsequent studies show that efficient countermeasures may retain exploitable structure or impose substantial overhead [7,8]. These trade-offs motivate reporting residual leakage together with bandwidth and latency costs rather than treating padding as a binary protection.

For generative AI, Weiss *et al.* demonstrated remote reconstruction from token-length patterns and introduced T5-based sequence-to-text inversion [1]. Related work has examined merged LLM API outputs, in which multiple tokens may share an observable unit [9], and encrypted-traffic inference against LLM interactions [10]. Relative to these studies, our contribution is not a new TLS extraction technique; it is a controlled cross-model benchmark of a fixed reconstructor, uncertainty-aware inference, and a defense pilot at the application-trace layer.

The reconstruction pipeline uses T5-style text-to-text transformers [11]. Evaluation combines normalized Levenshtein distance [12], ROUGE precision [13], and a single fixed term-frequency cosine score ϕ . Sentence-BERT [14] is discussed as a future paraphrase-aware measure, but its scores are not mixed with the final publication results.

3. Materials and Methods

3.1. Threat Model and Objective

We evaluate an attacker that receives the ordered byte lengths of SSE-equivalent application-layer events generated by an instrumented LLM service. The attacker is as-

sumed to observe only the lengths of streamed application events, without access to prompts, generated tokens, or internal model information. This is a length-oracle abstraction: unlike a fully passive network attacker, it does not need to infer application-event boundaries from TLS, HTTP/2, or QUIC traffic. The resulting measurements, therefore, characterize the information available once token-level lengths are observable. They do not address the separate problem of recovering those lengths from encrypted traffic in real-world deployments.

The primary objective is to determine how accurately assistant responses can be reconstructed from application-event lengths alone. The experimental campaign therefore addressed three tasks: (i) capture per-token side-channel traces during live inference, (ii) reconstruct text from the resulting length sequences, and (iii) quantify reconstruction quality against ground-truth outputs.

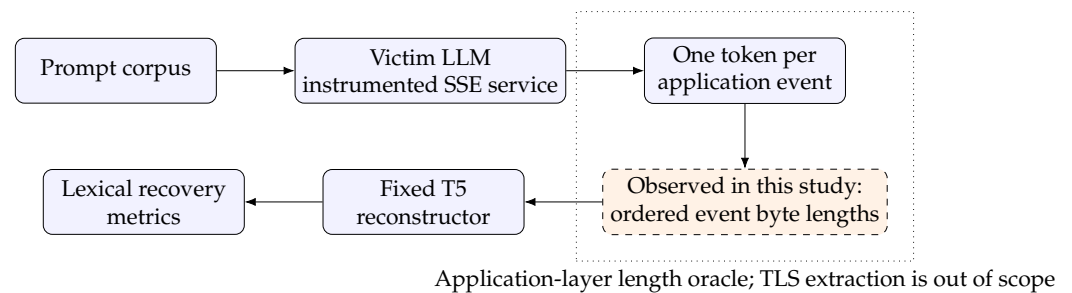


Figure 1. Threat model. The experiment begins at SSE-equivalent application-event lengths; it does not claim passive recovery of event boundaries from encrypted transport records.

3.2. Streaming Harness and Trace Collection

The measurement harness implemented SSE streaming in a strict one-token-per-event configuration. Each application event followed the format data: `<token>\n\n`, while stream termination used data: `[DONE]\n\n`. For every emitted token, the harness logged token text for ground-truth evaluation, token UTF-8 length, application-event UTF-8 length, and relative emission timestamp. Per-run metadata additionally included the model identifier, decoding parameters, prompt, generated response text, and token count. The attacker-facing reconstruction input contained lengths only.

Within the experimental harness, this design creates a direct mapping between emitted tokens and recorded SSE-equivalent application events. This mapping enables controlled reconstruction experiments from application-layer length traces. Denoting the UTF-8 length of the t -th recorded application event by L_t^{event} , the estimated token length is

$$L_t^{\text{tok}} = L_t^{\text{event}} - 8, \quad (1)$$

where 8 bytes correspond to the fixed overhead generated by the prefix data: `<token>` and suffix `\n\n` in the laboratory serialization. This relation applies only to the stated serialization, without additional SSE fields or transformations. The terminal `[DONE]` event is excluded from reconstruction.

3.3. Reconstruction Pipeline

The inversion pipeline follows the sequence-to-text procedure introduced by Weiss *et al.* [1]. First, the token-length sequence is segmented heuristically into sentence-like chunks. Second, the initial chunk is decoded using `royweiss1/T5_FirstSentences`. Third, subsequent chunks are decoded with `royweiss1/T5_MiddleSentences` using preceding context. Finally, multiple decoding candidates are ranked according to aggregate generation confidence represented by the transition log-probability sum.

The reconstruction design is conditional rather than purely tokenwise. Instead of recovering each token independently, it maps length patterns into coherent text spans and uses context to improve linguistic plausibility. Our quantitative claims nevertheless concern lexical recovery; semantic correctness was not established by human annotation.

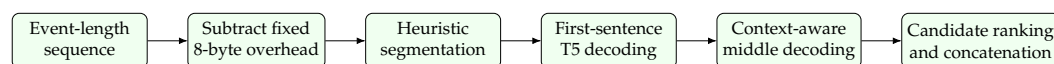


Figure 2. Reconstruction pipeline used for all victim models and defense transformations. The decoder checkpoints and ranking procedure remain fixed.

3.4. Prompt Corpus and Experimental Protocol

The prompt corpus comprised 15 topical files with 20 prompts each, yielding a total of 300 prompts. The corpus covered a broad range of potentially sensitive topics, including legal, health-related, financial, employment, and security matters. Prompt scheduling followed a round-robin strategy to reduce topical clustering effects during data collection.

Victim-model decoding was deterministic, with temperature set to 0.0 and top_p set to 1.0. For primary cross-model claims, a prespecified standardized subset was selected with the following shared parameters: $N = 300$, max_new_tokens=96, samples_per_segment=3, max_sentences=3, and num_first_candidates=3.

The evaluated models are listed in Table 1. Six are instruction- or chat-tuned; Phi-1.5 is a base model retained as a stress test [15–21]. A common token budget does not imply a common amount of generated text across tokenizer families. Table 2 reports response-length covariates computed from the 2100 publication samples. Truncation at the 96-token cap ranged from 95.0% (Qwen2.5-1.5B-Instruct) to 100% (four models), so the results primarily characterize reconstruction of truncated early-response content.

Table 1. Victim models used in the standardized comparison. Parameter counts follow the cited model cards and technical reports. Licenses are those published with the Hugging Face checkpoints used in the experiments.

Model	Checkpoint	Parameters	License	Tokenizer
Qwen2.5-1.5B-Instruct	Qwen/Qwen2.5-1.5B-Instruct	1.54B	Apache 2.0	Qwen2
Qwen2.5-3B-Instruct	Qwen/Qwen2.5-3B-Instruct	3.09B	Qwen Research	Qwen2
Llama-3.2-3B-Instruct	meta-llama/Llama-3.2-3B-Instruct	3.2B	Llama 3.2 Community	Llama 3
Gemma-2-2B-it	google/gemma-2-2b-it	2.6B	Gemma	Gemma
Phi-1.5	microsoft/phi-1.5	1.3B	MIT	CodeGen
TinyLlama-1.1B-Chat-v1.0	TinyLlama/TinyLlama-1.1B-Chat-v1.0	1.1B	Apache 2.0	Llama 2
Phi-3.5-mini-instruct	microsoft/Phi-3.5-mini-instruct	3.8B	MIT	Llama/Phi-3

Table 2. Response-length covariates for the standardized subset ($N = 300$ per model). Truncation is the percentage of samples with token_count ≥ 96 .

Model	Mean tokens	Mean chars	Mean B/token	Trunc. (%)
Qwen2.5-1.5B-Instruct	92.2	464	5.03	95.0
Qwen2.5-3B-Instruct	96.0	482	5.03	100.0
Llama-3.2-3B-Instruct	94.7	463	4.88	96.3
Gemma-2-2B-it	96.0	435	4.53	100.0
Phi-1.5	96.0	497	5.17	99.7
TinyLlama-1.1B-Chat-v1.0	96.0	331	3.45	100.0
Phi-3.5-mini-instruct	96.0	342	3.57	100.0

3.5. Evaluation Metrics

Reconstruction quality was computed using four complementary lexical criteria. Normalized Levenshtein distance (ed_norm) captures character-level divergence between the

reconstruction and reference [12]. ROUGE-1 precision and ROUGE-L precision quantify lexical overlap from unigram and longest-common-subsequence perspectives [13]. The final metric, ϕ , was recomputed for each of the 2100 samples as cosine similarity between term-frequency vectors of the reference and reconstruction, following the vector-space representation [22].

The study also reports lexical recovery rate LR@0.5, defined as the proportion of samples with $\phi > 0.5$. The threshold is a descriptive operating point and has not been calibrated against human judgments. Because the final publication analysis uses one term-frequency cosine backend, ϕ measures lexical overlap rather than paraphrase-aware semantic equivalence.

3.6. Statistical Analysis

Because the same 300 prompts were issued to every model, cross-model inference used prompt-aligned observations. Mean metrics were accompanied by percentile-bootstrap 95% confidence intervals [23] based on $B = 10,000$ paired-index resamples with replacement and seed 20260706. Confidence intervals for mean pairwise differences were obtained by bootstrapping the vector of within-prompt differences, thereby preserving pairing. Wilson 95% intervals were used for LR@0.5.

Pairwise differences in per-prompt ϕ were assessed using two-sided paired Wilcoxon signed-rank tests [24]. Absolute nonzero differences received average ranks when tied; exact zero differences were removed under the Wilcoxon convention. The implementation applied the standard tie correction to the normal-approximation variance and a continuity correction of 0.5. The effective n was 300 for 18 pairs, 299 for two pairs, and 219 for TinyLlama versus Phi-3.5-mini. Holm–Bonferroni correction controlled family-wise error over all $\binom{7}{2} = 21$ model pairs [25], and rank-biserial correlation quantified effect size. Defense–baseline comparisons used the same paired procedure with Holm correction over ten tests. Statistical significance was defined as adjusted $p < 0.05$.

3.7. Defense Evaluation Protocol

Five configurations were evaluated on the same 30 topic-balanced prompts for each of Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct. Let ℓ_t denote the observable byte length of event t before transformation.

- bucket_8: replace ℓ_t by $8\lceil\ell_t/8\rceil$, independently and online for every event.
- pad_32: emit an online constant-width 32-byte protected payload unit; the evaluated implementation retains the application-event structure and accounts for bytes required to reach that unit.
- batch_2 and batch_4: concatenate the lengths of consecutive groups of two or four tokens into one attacker-visible value. The final incomplete group, if present, is retained.
- rand_pad_8: add an independently sampled integer $U_t \sim \text{Uniform}\{0, \dots, 8\}$ using a recorded deterministic seed.

The transformed length sequences were passed directly to the same segmentation and decoding pipeline. The reconstructor was frozen and was not retrained or calibrated on defended traces. Defense effectiveness was defined as $100(1 - \bar{\phi}_d/\bar{\phi}_0)$. Application-byte overhead was computed from transformed versus original application payload-length sums. Batching has zero overhead under this definition, which excludes record headers, buffering delay, and user-perceived latency. Mean observable frame count captures the structural effect of batching.

3.8. Run Completeness Criteria

A run was considered complete only if all required artifacts were present (`config.json`, `progress.json`, `samples.jsonl`, `summary.json`, `summary.md`, and `summary_by_topic.csv`) and internal counts matched exactly:

$$\text{progress.done} = \text{progress.total} = \text{summary.count} = \text{lines}(\text{samples.jsonl}). \quad (2)$$

These checks were applied before including any run in the main comparative analysis.

3.9. Reproducibility Artifacts

All experimental materials required to reproduce the reported tables and figures are released in the public repository at <https://github.com/jackaugustyn/LLeakM>. The package includes the instrumented SSE server, the reconstruction pipeline, the 300-prompt corpus, per-sample outputs for the seven standardized runs, defense and trace-baseline results, analysis scripts, and a SHA-256 manifest under `experiment_validation/`.

4. Results

4.1. Completeness Audit

The experiment directory contained 20 candidate execution folders, of which 14 were artifact-complete and 6 were incomplete placeholders. The complete runs contained 4050 samples in total. The prespecified standardized subset used for primary comparison covered seven models and 2100 samples. Each included model contributed exactly 300 samples. The corresponding artifact directories are listed in Appendix A.

The artifact-completeness audit supports model-level comparison under the specified homogeneous settings. The incomplete placeholders also demonstrate the importance of explicit completeness filters before aggregating results across runs.

4.2. Cross-Model Comparison under Standardized Conditions

Table 3 reports the primary comparable subset using the fixed publication backend. The quality of reconstruction varied substantially between the evaluated models. The two Qwen2.5 variants consistently achieved the highest lexical recovery scores, followed by Phi-1.5 and Llama-3.2-3B-Instruct. Gemma-2-2B-it occupied an intermediate position, while Phi-3.5-mini-instruct and TinyLlama-1.1B-Chat-v1.0 exhibited only limited reconstructability in the evaluated pipeline. These differences suggest that the relationship between length patterns and generated content is not uniform across model families. Although Phi-1.5 achieved relatively strong results, it should be interpreted separately because it is a base model rather than an instruction-tuned assistant. Consequently, its performance does not imply that base models are inherently more vulnerable than instruction-following systems.

Table 3. Primary cross-model comparison for the standardized subset using the fixed publication term-frequency cosine backend ($N = 300$).

Model	Type	ϕ_{mean}	ed_{norm}	R1 Precision	RL Precision	LR@0.5
Qwen/Qwen2.5-1.5B-Instruct	instruct	0.3242	0.7199	0.3220	0.2651	8.667%
Qwen/Qwen2.5-3B-Instruct	instruct	0.3096	0.7300	0.2945	0.2345	3.000%
microsoft/phi-1_5	base	0.2747	0.7426	0.2503	0.1891	4.333%
meta-llama/Llama-3.2-3B-Instruct	instruct	0.2538	0.7499	0.2387	0.1824	3.000%
google/gemma-2-2b-it	instruct	0.1798	0.7764	0.1719	0.1269	0.000%
TinyLlama/TinyLlama-1.1B-Chat-v1.0	chat	0.0428	0.8026	0.0312	0.0310	0.000%
microsoft/Phi-3.5-mini-instruct	instruct	0.0239	0.8087	0.0158	0.0157	0.000%

The historical per-run file for Phi-1.5 contains $\phi = 0.0575$ and a 0.333% threshold rate because it was generated with the backend available during that run. Those values are

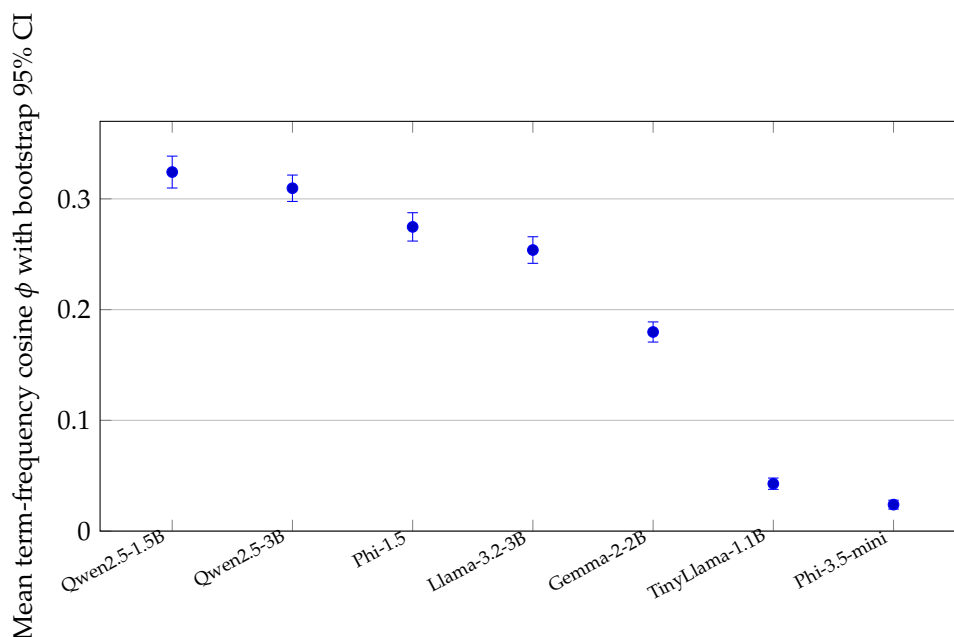


Figure 3. Mean lexical reconstruction score and bootstrap 95% confidence intervals for the seven standardized runs. Error-bar half-widths are rounded to four decimals for visualization; asymmetric endpoints are reported in Table 4.

not used in this manuscript. All tables, confidence intervals, and hypothesis tests use the uniformly rescored per-prompt publication dataset, for which Phi-1.5 has mean $\phi = 0.2747$ and LR@0.5 = 4.33%.

4.3. Statistical Uncertainty and Pairwise Tests

Table 4 augments point estimates with uncertainty intervals. Nineteen of 21 paired cross-model comparisons remained significant after Holm correction. The two exceptions were Qwen2.5-1.5B versus Qwen2.5-3B (mean paired difference 0.0146, unadjusted bootstrap 95% CI $[-0.0026, 0.0314]$, rank-biserial $r = 0.089$, $p_{\text{Holm}} = 0.1838$) and Llama-3.2-3B versus Phi-1.5 (difference -0.0209 , unadjusted bootstrap 95% CI $[-0.0375, -0.0043]$, $r = -0.148$, $p_{\text{Holm}} = 0.0533$). In contrast, Qwen2.5-1.5B versus Phi-3.5-mini showed a large difference (0.3002, $r = 1.000$, $p_{\text{Holm}} < 0.0001$). Thus, most observed gaps are distinguishable from prompt-sampling variation in this benchmark, but the tests do not establish causal model effects. An ordinary-least-squares check on the six instruction/chat models ($n = 1800$) found that model indicators alone explained $R^2 = 0.642$ of per-prompt ϕ , compared with $R^2 = 0.509$ for response characters, mean token bytes, and truncation, and $R^2 = 0.646$ for the combined specification. Length covariates therefore do not account for the observed ranking.

Table 4. Uncertainty-aware results. Mean- ϕ intervals use 10,000 bootstrap resamples; LR@0.5 intervals are Wilson 95% intervals.

Model	Mean ϕ	95% CI	LR@0.5	95% CI
Qwen2.5-1.5B-Instruct	0.3242	[0.3097, 0.3380]	8.67%	[5.98, 12.40]%
Qwen2.5-3B-Instruct	0.3096	[0.2977, 0.3214]	3.00%	[1.59, 5.60]%
Phi-1.5	0.2747	[0.2622, 0.2877]	4.33%	[2.55, 7.27]%
Llama-3.2-3B-Instruct	0.2538	[0.2419, 0.2658]	3.00%	[1.59, 5.60]%
Gemma-2-2B-it	0.1798	[0.1708, 0.1889]	0.00%	[0.00, 1.26]%
TinyLlama-1.1B-Chat	0.0428	[0.0377, 0.0479]	0.00%	[0.00, 1.26]%
Phi-3.5-mini-instruct	0.0239	[0.0201, 0.0280]	0.00%	[0.00, 1.26]%

4.4. Focused Comparison of Three Model Families

Table 5 isolates Llama-3.2-3B-Instruct, Gemma-2-2B-it, and Phi-3.5-mini-instruct as a focused cross-family comparison. Llama-3.2-3B-Instruct was the most reconstructable of the three, with $\phi = 0.2538$ and a nonzero threshold recovery rate. Gemma-2-2B-it occupied a middle position with lower lexical scores. Phi-3.5-mini-instruct yielded the lowest scores for this fixed pipeline, with near-zero lexical similarity and no samples above $\phi > 0.5$.

Because all three runs contain the planned 300 prompt-aligned observations, the reported ordering cannot be explained by differences in run completeness. Within the scope of the present pipeline, the observed reconstruction ordering is Llama-3.2-3B-Instruct > Gemma-2-2B-it > Phi-3.5-mini-instruct.

Table 5. Focused results for three victim-model families.

Model	Completion	ϕ_{mean}	ed_{norm}	R1	RL	LR@0.5
Llama-3.2-3B-Instruct	300/300	0.2538	0.7499	0.2387	0.1824	3.000%
Phi-3.5-mini-instruct	300/300	0.0239	0.8087	0.0158	0.0157	0.000%
Gemma-2-2B-it	300/300	0.1798	0.7764	0.1719	0.1269	0.000%

4.5. Exploratory Topic Sensitivity

All seven standardized runs retain a topic label on every sample. The present topic-level analysis is nevertheless descriptive: no multiplicity-corrected inference was performed over model–topic combinations. Selected topic-level summaries are shown in Table 6. For Llama-3.2-3B-Instruct, the highest reported topic mean was for *employment* ($\phi = 0.3373$), and the lowest was for *safety_security* ($\phi = 0.2053$). Gemma-2-2B-it had the highest reported mean for *beliefs_values* and the lowest for *financial_status*. Phi-3.5-mini-instruct remained low across its reported topic extrema.

These descriptive observations suggest that reconstruction scores may depend on the linguistic distribution of generated content. They do not support confirmatory topic-level claims.

Table 6. Selected topic-level sensitivity based on mean fixed-backend lexical similarity.

Model	Highest reported topic	ϕ_{mean}	Lowest reported topic	ϕ_{mean}
Llama-3.2-3B-Instruct	employment	0.3373	safety_security	0.2053
Gemma-2-2B-it	beliefs_values	0.2090	financial_status	0.1575
Phi-3.5-mini-instruct	financial_status	0.0444	personal_identity	0.0073

4.6. Effectiveness of Defensive Mechanisms

All evaluated defenses reduced reconstruction quality, although the magnitude of protection differed considerably between approaches. Among the tested mechanisms, byte bucketing provided the strongest reduction in observable leakage, while batching achieved substantial improvements without increasing application-payload size under the adopted accounting model. Randomized padding offered a moderate reduction in recovery quality, whereas fixed-width framing imposed considerable overhead while providing more limited protection.

None of the defended configurations produced examples with high lexical recovery. Although the evaluation set is too small to exclude residual risk entirely, the results indicate that relatively simple modifications of observable length patterns can significantly hinder reconstruction by the evaluated attacker.

Table 7. Defense effectiveness against the frozen, non-adaptive reconstructor ($n = 30$ paired prompts per model). Application-byte overhead excludes transport headers, buffering delay, and latency.

Model	Defense	Mean ϕ	95% CI	Reduction	App.-byte overhead	Frames
Qwen2.5-1.5B	baseline	0.3010	[0.2537, 0.3508]	–	0.0%	96
	bucket_8	0.0293	[0.0198, 0.0397]	90.3%	82.4%	96
	pad_32	0.1901	[0.1667, 0.2150]	36.8%	538.0%	96
	batch_2	0.0928	[0.0791, 0.1072]	69.2%	0.0%	48
	batch_4	0.0950	[0.0668, 0.1269]	68.4%	0.0%	24
	rand_pad_8	0.1033	[0.0871, 0.1203]	65.7%	78.5%	96
Llama-3.2-3B	baseline	0.2576	[0.2177, 0.2978]	–	0.0%	96
	bucket_8	0.0428	[0.0305, 0.0563]	83.4%	88.6%	96
	pad_32	0.1674	[0.1382, 0.2003]	35.0%	565.1%	96
	batch_2	0.0823	[0.0684, 0.0967]	68.0%	0.0%	48
	batch_4	0.0832	[0.0615, 0.1055]	67.7%	0.0%	24
	rand_pad_8	0.1004	[0.0834, 0.1186]	61.0%	81.9%	96

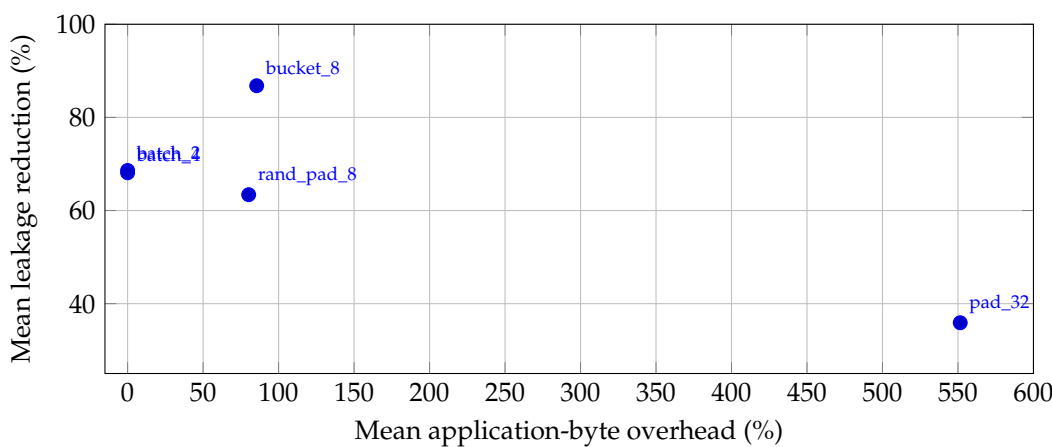


Figure 4. Defense effectiveness versus mean application-byte overhead, averaged over the two evaluated models. Batching overlaps at zero byte-sum overhead but changes frame count and latency, which are not represented on the horizontal axis.

5. Discussion272

5.1. Interpretation of Leakage Differences273

Reconstruction outcomes differed across the evaluated models under identical col-274
lection and reconstruction settings. This finding has practical implications only after275
token-granular application-event lengths are available. A service that encrypts SSE pay-276
loads at the transport layer is not thereby shown to be safe against reconstruction, but277
neither do the present measurements demonstrate that those lengths can be recovered from278
TLS, HTTP/2, or QUIC records.279

Although reconstruction performance differed substantially across models, the reasons280
behind these differences remain unclear. Several factors may contribute to the observed281
variation, including tokenization schemes, response style, output-length distributions,282
and the degree of compatibility between model outputs and the reconstruction pipeline.283
Disentangling these effects will require dedicated ablation studies and remains an important284
direction for future work.285

5.2. Practical Exploitability286

The threshold results indicate that high-overlap lexical reconstruction is not universal.287
The observed recovery rates nevertheless indicate the potential for practical exploitation288
in some length-oracle configurations, although production evaluation is needed. Even289
moderate lexical similarity may warrant attention when generated content concerns sen-290

sitive domains. An adversary may also combine repeated observations, additional side information, or adaptive re-ranking, none of which was evaluated here.

Attack efficacy, operational exploitability, and defense efficacy should be reported separately. Mean lexical similarity captures aggregate word overlap but does not establish semantic correctness or actionability. Future studies should combine lexical scores with a fixed paraphrase-aware metric and blinded human judgments.

5.3. Limitations

As with any controlled experimental study, the reported findings should be interpreted within the scope of the adopted methodology. The primary comparison focuses on a standardized subset of models and evaluates lexical recovery rather than semantic understanding. Although the released artifacts contain both reference and reconstructed texts, the present analysis does not include blinded human evaluation or uniformly computed paraphrase-aware metrics. In addition, deterministic decoding was chosen to improve experimental control, but it does not capture the full range of deployment settings encountered in practice. The defense evaluation covers only two victim models and assumes a non-adaptive attacker that is neither retrained nor optimized on defended traces. The 96-token generation budget caused most responses to be truncated; consequently, the reported results characterize early-response reconstruction and may not generalize to complete, longer answers.

Several important aspects remain outside the scope of the present work, including reconstruction variability across multiple random seeds, end-to-end extraction from encrypted transport records, latency effects, protocol-header overhead, and user-experience costs. The results should therefore be viewed as a controlled baseline for understanding leakage potential rather than as a complete characterization of real-world streaming side-channel risk.

5.4. Ethical Considerations

This work is positioned as defensive security research. Its goal is to quantify leakage in order to motivate safer streaming implementations and more realistic privacy assessments for AI assistants. Because the attack concerns potentially sensitive outputs, public release of artifacts should be handled responsibly, with appropriate redaction, disclosure timing, and inclusion of mitigations.

6. Conclusions

This study examined the extent to which token-length information can reveal characteristics of generated content in streaming LLM systems. Using a Weiss-style reconstruction pipeline and an artifact-validated dataset of SSE-equivalent application-event lengths, we observed measurable lexical leakage across a diverse set of open-weight models, with reconstruction performance varying substantially between model families.

The results show that leakage is not uniform across models and that simple modifications of observable length patterns can significantly reduce reconstruction quality. While the evaluation assumes access to token-granular application-layer traces, it highlights the broader privacy implications of metadata leakage in modern AI systems.

Taken together, these findings suggest that streaming architectures deserve closer attention from both researchers and system designers. Understanding how seemingly innocuous metadata can expose information about generated content will be essential for developing more privacy-preserving LLM deployments.

Future work should extend the analysis to adaptive attackers, transport-layer extraction, paraphrase-aware semantic evaluation, and human assessment of reconstruc-

tion quality. Such investigations will be necessary before making stronger claims about deployment-scale exploitability or defense effectiveness in real-world environments.

Appendix A. Artifact Completeness and Standardized Run Directories

For reproducibility, a run should be included in comparative analyses only if all required files are present and all internal sample counts match exactly. In the present campaign, the required files were `config.json`, `progress.json`, `samples.jsonl`, `summary.json`, `summary.md`, and `summary_by_topic.csv`. Placeholder or partially completed execution folders were excluded from primary cross-model claims.

Table A1 lists the directory names of the seven standardized runs in the public artifact package at <https://github.com/jackaugustyn/LLeakM>. These identifiers are retained because they are the paths needed to locate the exact per-sample files, checksums, and analysis inputs used in this manuscript.

Table A1. Public artifact directories for the standardized publication subset.

Model	Artifact directory
Qwen2.5-1.5B-Instruct	run_20260325_104518_qwen_1_5b
Qwen2.5-3B-Instruct	run_20260327_124817_qwen_3b_final_resume
Phi-1.5	run_20260329_085714_phi_1_5
Llama-3.2-3B-Instruct	run_20260414_073030_llama_3_2_3b
Gemma-2-2B-it	run_20260415_080034_gemma_2_2b_it
TinyLlama-1.1B-Chat-v1.0	run_20260325_153010_tinyllama_1_1b
Phi-3.5-mini-instruct	run_20260414_111644_phi_3_5_mini

Author Contributions: Conceptualization, P.A. and S.J.; methodology, P.A.; software, P.A.; validation, P.A.; formal analysis, P.A. and S.J.; investigation, P.A.; data curation, P.A.; writing—original draft preparation, P.A.; writing—review and editing, P.A. and S.J.; supervision, S.J. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable. This study evaluated software systems and generated model outputs and did not involve human participants or identifiable personal data.

Informed Consent Statement: Not applicable.

Data Availability Statement: The code, prompt corpus, per-sample measurements, aligned publication metrics, statistical outputs, defense results, analysis scripts, and reproducibility manifest presented in this study are openly available at <https://github.com/jackaugustyn/LLeakM> in the directory `experiment_validation/`.

Conflicts of Interest: The authors declare no conflict of interest.

References

1.

Weiss, R.; Ayzenshteyn, D.; Amit, G.; Mirsky, Y. What Was Your Prompt? A Remote Keylogging Attack on AI Assistants. In Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, 2024.

364
365
366

2.

Kocher, P.C. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In Proceedings of the Advances in Cryptology — CRYPTO’96. Springer, 1996, pp. 104–113.

367
368
369

3.

Song, D.X.; Wagner, D.; Tian, X. Timing Analysis of Keystrokes and Timing Attacks on SSH. In Proceedings of the 10th USENIX Security Symposium (USENIX Security 01). USENIX Association, 2001.

370
371
372

4. Chen, S.; Wang, R.; Wang, X.; Zhang, K. Side-Channel Leaks in Web Applications: A Reality Today, a Challenge Tomorrow. In Proceedings of the 2010 IEEE Symposium on Security and Privacy. IEEE, 2010, pp. 191–206. 373–375
5. Cai, X.; Zhang, X.C.; Joshi, B.; Johnson, R. Touching from a Distance: Website Fingerprinting Attacks and Defenses. In Proceedings of the Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS). ACM, 2012, pp. 605–616. 376–378
6. Sirinam, P.; Imani, M.; Juarez, M.; Wright, M. Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning. In Proceedings of the Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, 2018, pp. 1928–1943. 379–382
7. Wright, C.V.; Coull, S.E.; Monrose, F. Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis. In Proceedings of the Proceedings of the Network and Distributed System Security Symposium (NDSS), 2009. 383–385
8. Dyer, K.P.; Coull, S.E.; Ristenpart, T.; Shrimpton, T. Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail. In Proceedings of the 2012 IEEE Symposium on Security and Privacy. IEEE, 2012, pp. 332–346. 386–388
9. Li, S.; Cui, T.; Chen, M.; Lin, X.; Gu, Z.; Deng, X.; Xu, K.; Li, Q. From Length to Content: Token-Length Side-Channel Attacks on Merged LLM API Outputs. In Proceedings of the 35th USENIX Security Symposium (USENIX Security 26). USENIX Association, 2026. Accepted for publication. 389–392
10. McDonald, G.; Bar Or, J. Whisper Leak: A Side-Channel Attack on Large Language Models. *arXiv preprint arXiv:2511.03675* **2025**, [arXiv:cs.CR/2511.03675]. 393–394
11. Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; Liu, P.J. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* **2020**, *21*, 1–67. 395–397
12. Levenshtein, V.I. Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. *Soviet Physics Doklady* **1966**, *10*, 707–710. 398–399
13. Lin, C. ROUGE: A Package for Automatic Evaluation of Summaries. In Proceedings of the Text Summarization Branches Out: Proceedings of the ACL-04 Workshop, 2004, pp. 74–81. 400–401
14. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics, 2019, pp. 3982–3992. <https://doi.org/10.18653/v1/D19-1410>. 402–406
15. Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* **2024**. 407
16. Dubey, A.; et al. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* **2024**. 408
17. Meta AI. Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>, 2024. Accessed 19 September 2026. 409–411
18. Gemma Team. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv preprint arXiv:2408.00118* **2024**. 412–413
19. Li, Y.; Bubeck, S.; Eldan, R.; Del Giorno, A.; Gunasekar, S.; Lee, Y.T. Textbooks Are All You Need II: phi-1.5 Technical Report. *arXiv preprint arXiv:2309.05463* **2023**. 414–415
20. Zhang, P.; Zeng, G.; Wang, T.; Lu, W. TinyLlama: An Open-Source Small Language Model. *arXiv preprint arXiv:2401.02385* **2024**. 416–417
21. Abdin, M.; Jacobs, S.A.; Awan, A.A.; Aneja, J.; Awadallah, A.; Awadalla, H.; Bach, N.; Bahree, A.; Bakhtiari, A.; Behl, H.; et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. *arXiv preprint arXiv:2404.14219* **2024**. 418–420
22. Salton, G.; Wong, A.; Yang, C.S. A Vector Space Model for Automatic Indexing. *Communications of the ACM* **1975**, *18*, 613–620. <https://doi.org/10.1145/361219.361220>. 421–422
23. Efron, B.; Tibshirani, R.J. *An Introduction to the Bootstrap*; Chapman & Hall/CRC, 1994. 423
24. Wilcoxon, F. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* **1945**, *1*, 80–83. 424
25. Holm, S. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* **1979**, *6*, 65–70. 425–426